

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 7
по дисциплине «Анализ алгоритмов»

Тема Конечные автоматы

Студент Атаев К. А.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л., Строганов Д.В

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Особенности анализа PDF-документов 5

1.2 Условия ошибки в PDF-документах 5

1.3 Применение регулярных выражений 5

1.4 Использование больших языковых моделей (БЯМ) 6

1.5 Краткое описание используемых БЯМ 6

1.6 Вывод 6

2 Конструкторская часть **7**

2.1 Общая структура программного обеспечения 7

2.2 Модель координат 7

2.3 Использование регулярных выражений 7

2.4 Логика проверки варианта 8

2.5 Организация взаимодействия с БЯМ 8

2.6 Анализ разработанных решений 8

2.7 Листинги ключевых алгоритмов 9

2.8 Вывод 9

3 Технологическая часть **10**

3.1 Средства реализации 10

3.1.1 Организация итоговой программы 10

3.2 Взаимодействие с БЯМ и итеративная доработка 10

3.2.1 Попытки с ChatGPT (GPT-5.2) 10

3.2.2 Попытки с DeepSeek DeepThink 11

3.2.3 Попытки с Qwen Coder 11

3.3 Пример запуска и получаемые файлы 12

3.4 Вывод 12

4 Исследовательский раздел **13**

4.1 Форма представления результатов 13
4.1.1 Результаты тестирования (реализация ChatGPT / GPT-5.2) 13
4.1.2 Результаты тестирования (реализация DeepSeek DeepThink) 14
4.1.3 Результаты тестирования (реализация Qwen Coder) 14
4.2 Сравнение результатов трёх реализаций 15
4.3 Оценка сложности 15
4.4 Вывод 15

ЗАКЛЮЧЕНИЕ **16**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **17**

ПРИЛОЖЕНИЕ А **18**

ВВЕДЕНИЕ

В настоящее время автоматическая проверка структуры текстовых документов является актуальной задачей, особенно в образовательной сфере, где к оформлению отчётов, курсовых и выпускных квалификационных работ предъявляются строгие требования. Одним из распространённых форматов хранения таких документов является PDF, который затрудняет автоматический анализ структуры.

Целью данной лабораторной работы является разработка программного обеспечения на языке Python для извлечения и анализа текстовых данных из PDF-файлов с использованием библиотеки PyPDF2 и регулярных выражений. Вариант задания заключается в проверке корректности наличия структурных элементов «ВВЕДЕНИЕ», «ЗАКЛЮЧЕНИЕ», «СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ» в тексте документа и их присутствия в содержании, а также в выявлении ошибочных наименований указанных элементов.

Для достижения поставленной цели в работе необходимо решить следующие задачи:

— разработать регулярные выражения для проверки структурных элементов документа согласно варианту;

— реализовать функцию поиска подстрок в PDF-файле с использованием регулярных выражений;

— разработать программное обеспечение, принимающее на вход путь к PDF-файлу или каталогу с файлами;

— выполнить тестирование программы на наборе PDF-файлов, приложенных к лабораторной работе;

— провести сравнение решений, полученных с использованием различных больших языковых моделей.

1 Аналитическая часть

В данном разделе будет рассмотрена постановка задачи, особенности анализа PDF-документов, а также теоретические основы применения регулярных выражений и больших языковых моделей (БЯМ) для разработки программного обеспечения.

1.1 Особенности анализа PDF-документов

Формат PDF ориентирован на точное отображение документа при печати и просмотре, поэтому он не хранит логическую структуру в привычном виде (разделы, заголовки, оглавление), а представляет документ как набор страниц с текстовыми и графическими блоками. Это осложняет автоматическую проверку структуры: один и тот же визуальный результат может соответствовать различным внутренним представлениям.

Для решения задачи используется библиотека PyPDF2, позволяющая извлекать текст постранично. После извлечения текста анализ выполняется на уровне строк, что позволяет ввести упрощённые координаты обнаруженных фрагментов в виде пары «номер страницы — номер строки». Такой подход соответствует требованиям задания и не требует вычисления точных геометрических координат текстовых блоков.

1.2 Условия ошибки в PDF-документах

Согласно варианту задания, документ считается содержащим ошибку в следующих случаях:

— структурный элемент отсутствует в основном тексте документа;

— структурный элемент отсутствует в содержании (оглавлении);

— используется ошибочное наименование структурного элемента (например, «Введение» вместо «ВВЕДЕНИЕ», «Литература» вместо «СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ», «Библиография», «Источники» и т.п.);

1.3 Применение регулярных выражений

Для автоматизации проверки используются регулярные выражения. Регулярные выражения являются формальным инструментом для описания шаблонов строк и поиска совпадений в тексте. Их преимущество заключается в детерминированности результата: при фиксированных правилах поиска программа выдаёт воспроизводимый итог без неоднозначной интерпретации. В то же время при анализе PDF необходимо учитывать, что извлечённый текст может содержать лишние пробелы, переносы, разрывы слов и другие артефакты конвертации. Поэтому при разработке шаблонов поиска закладывается устойчивость к вариативности регистра, пробельных символов и возможных форматных искажений.

1.4 Использование больших языковых моделей (БЯМ)

В рамках выполнения лабораторной работы применялись большие языковые модели (БЯМ). БЯМ представляют собой нейросетевые модели, обученные на больших корпусах текстов, которые способны генерировать связный текст, объяснять решения и предлагать программный код по описанию задачи. Практическая ценность БЯМ в данной работе заключается в ускорении этапов проектирования: модели помогают формулировать требования, предлагать архитектуру решения, генерировать черновые реализации и указывать возможные крайние случаи.

1.5 Краткое описание используемых БЯМ

В рамках разработки решения были использованы три большие языковые модели:

— **ChatGPT (GPT-5.2)** — модель общего назначения, которая используется для уточнения требований, построения структуры решения и получения развёрнутых объяснений по алгоритму. Эта модель эффективно помогает на этапе генерации общей структуры программы, а также в уточнении задач и требований.

— **DeepSeek DeepThink** — модель, ориентированная на последовательное рассуждение и проработку логики проверки, включая формулирование правил обнаружения ошибок. Она используется для создания устойчивых алгоритмов поиска структурных элементов в тексте, а также для выявления ошибок.

— **Qwen Coder** — модель, специализирующаяся на генерации программного кода и типовой инженерной структуре (разделение ядра алгоритма, интерфейса командной строки и модулей вывода результатов). Модель Qwen Coder полезна на этапе реализации алгоритма, когда требуется быстро сгенерировать программный код, который затем можно доработать и адаптировать под конкретные требования.

1.6 Вывод

В этом разделе были рассмотрены особенности анализа PDF-документов и выбран подход к определению координат обнаруженных фрагментов в формате «номер страницы — номер строки». Были сформулированы условия, при которых документ считается содержащим ошибку согласно варианту, а также обосновано применение регулярных выражений для поиска структурных элементов.

2 Конструкторская часть

В данном разделе рассматривается устройство разрабатываемого программного обеспечения. Описывается разбиение системы на основные модули, используемые модели данных, а также порядок взаимодействия с выбранными большими языковыми моделями при решении задачи проверки структуры PDF-документов.

2.1 Общая структура программного обеспечения

Проектируемое программное обеспечение состоит из следующих логических компонентов:

1) модуль извлечения текстового содержимого из PDF-файла с сохранением координат строк (номер страницы и номер строки);
2) модуль определения границ области «СОДЕРЖАНИЕ/ОГЛАВЛЕНИЕ»;
3) модуль проверки выполнения условий варианта на основе регулярных выражений;
4) модуль фиксации первого обнаруженного нарушения с указанием координат;
5) модуль пакетной обработки PDF-файлов и формирования итогового отчёта в формате CSV.

Такая модульная структура упрощает разработку, отладку и последующее расширение программного обеспечения.

2.2 Модель координат

В качестве координат найденных совпадений используется пара значений: номер страницы PDF-документа и номер строки на данной странице. Оба значения нумеруются с единицы.

Использование данной модели координат позволяет однозначно указать место обнаружения нарушения в документе без необходимости анализа геометрической структуры PDF-файла, что соответствует требованиям задания.

2.3 Использование регулярных выражений

Для поиска и проверки структурных элементов документа применяются регулярные выражения, учитывающие следующие особенности:

— вариативность регистра символов;
— возможные пробелы между буквами в заголовках (например, «В В Е Д Е Н И Е»);
— наличие номерных префиксов («1», «1.2»), располагающихся перед заголовком;
— наличие ошибочных или недопустимых наименований (синонимы и близкие формулировки).

Данный подход позволяет корректно обрабатывать документы, оформленные с отклонениями от шаблона, но сохраняющие требуемую структуру.

2.4 Логика проверки варианта

Алгоритм проверки одного PDF-файла включает следующие этапы:

1) извлечение текстовых строк документа с сохранением их координат (номер страницы, номер строки);

2) поиск границ области оглавления по заголовку «СОДЕРЖАНИЕ/ОГЛАВЛЕНИЕ» и характерной форме строк-элементов;

3) проверку наличия в оглавлении строк «ВВЕДЕНИЕ», «ЗАКЛЮЧЕНИЕ», «СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ»;

4) проверку отсутствия корректных заголовков указанных разделов в основном тексте документа;

5) проверку отсутствия ошибочных наименований разделов (например, «Вступление», «Выводы», «Литература», «Библиография») во всём документе;

6) при обнаружении первого нарушения — фиксацию координат проблемной строки и завершение проверки файла.

2.5 Организация взаимодействия с БЯМ

В рамках выполнения работы большие языковые модели использовались для получения альтернативных реализаций алгоритма проверки структуры PDF-документов. Взаимодействие с каждой моделью осуществлялось в виде текстовых запросов, содержащих описание условий варианта и требований к логике проверки.

Для всех моделей использовался одинаковый набор требований, что позволило корректно сравнить полученные решения. Результаты работы каждой модели представлены в виде программных реализаций, приведённых в приложении А.

2.6 Анализ разработанных решений

Реализация, полученная с использованием ChatGPT (GPT-5.2), приведена в листинге А.1. Данная модель предложила наиболее полную и структурированную реализацию, включающую извлечение текста из PDF-файла, формирование набора регулярных выражений и базовые функции обработки строк. Основное достоинство данного решения заключается в читаемости кода и чётком разделении этапов обработки документа.

Реализация, полученная с использованием модели DeepSeek DeepThink, представлена в листинге А.2. В этом решении основной акцент сделан на алгоритм определения границ области оглавления и проверку структуры документа. Подход отличается более строгой логикой проверки и повышенной устойчивостью к нетипичному форматированию PDF-файлов.

Реализация, полученная с использованием модели Qwen Coder, приведена в листинге А.3. Данное решение отличается использованием оптимизации раннего выхода из алгоритма при обнаружении всех обязательных элементов структуры. Это позволяет сократить количество

проверок и уменьшить время обработки документа в ряде случаев.

2.7 Листинги ключевых алгоритмов

Полные реализации алгоритмов имеют значительный объём, поэтому в приложении приведены укороченные листинги, содержащие ядро логики проверки структуры документа без вспомогательного ввода-вывода и интерфейсных компонентов.

— Листинг А.1 — реализация, полученная с использованием ChatGPT / GPT-5.2;

— Листинг А.2 — реализация, полученная с использованием DeepSeek DeepThink;

— Листинг А.3 — реализация, полученная с использованием Qwen Coder.

2.8 Вывод

В данном разделе была рассмотрена архитектура программного обеспечения, описана логика проверки структуры PDF-документов, а также проанализированы решения, полученные с использованием различных больших языковых моделей.

3 Технологическая часть

В этом разделе будет описан процесс разработки программного обеспечения: используемые средства, последовательность взаимодействия с большими языковыми моделями (БЯМ), исходные промпты, полученные результаты, внесённые правки и их итоговый эффект.

3.1 Средства реализации

Для реализации использовался язык программиров*ания Python 3.10. Основные библиотеки:

— PyPDF2 — извлечение текста из PDF;

— re — регулярные выражения;

— csv — сохранение результатов в формате CSV;

— pathlib — работа с путями и каталогами;

— argparse — интерфейс командной строки.

3.1.1 Организация итоговой программы

Во всех финальных версиях программ выделены две части:

— **алгоритм (ядро)** — извлечение строк, выделение оглавления, проверка правил варианта, вычисление координат первого нарушения;

— **вывод и CLI** — чтение аргументов, пакетная обработка файлов и экспорт CSV.

Такое разделение использовано для того, чтобы алгоритм можно было переиспользовать как функцию в других программах, а формат вывода (CSV или печать в консоль) изменять независимо.

3.2 Взаимодействие с БЯМ и итеративная доработка

Разработка велась в итеративном режиме. На каждом шаге формулировался промпт с требованиями варианта, полученный код проверялся на приложенных PDF-файлах, после чего фиксировались расхождения и формулировались уточняющие запросы либо выполнялись ручные правки.

Ниже приведены основные попытки для каждой из трёх БЯМ.

3.2.1 Попытки с ChatGPT (GPT-5.2)

Промпт 1.

Сгенерируй Python-программу с использованием PyPDF2 и регулярных выражений. Вариант: проверить отсутствие структурных элементов ВВЕДЕНИЕ, ЗАКЛЮЧЕНИЕ, СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ в тексте документа и их

присутствие в содержании, а также отсутствие ошибочных наименований. Функция должна возвращать (bool, список кортежей (строка, (страница, строка))).

Результат. Была получена базовая реализация с постраничным извлечением текста и поиском совпадений. Функция возвращала булев результат и список совпадений.

Проблема. При запуске на тестовых PDF выявилась необходимость:

— учитывать «растянутые» заголовки (пример: `В В Е Д Е Н И Е`);

— корректно выделять область оглавления, а не искать элементы по всему документу;

— фиксировать координату *первого нарушения* (а не только успешные совпадения).

Правка 1. Добавлены регулярные выражения с допуском пробелов между буквами (через шаблон вида `V\s*V\s*E...`), а также реализовано выделение области оглавления по заголовку `СОДЕРЖАНИЕ/ОГЛАВЛЕНИЕ` и форме строк-элементов оглавления.

Результат правки 1. Улучшилась устойчивость к артефактам извлечения текста, а также появилась корректная проверка «в оглавлении / в тексте».

Правка 2. Реализован режим пакетной обработки каталога и экспорт результатов в CSV. Вывод унифицирован под требование: **«найдена ошибка» / «ошибка не найдена»**.

Результат правки 2. Программа стала формировать таблицу результатов автоматически, что позволяет напрямую использовать CSV в исследовательском разделе отчёта.

3.2.2 Попытки с DeepSeek DeepThink

Промпт 1.

Сгенерируй альтернативную реализацию того же варианта проверки структуры PDF. Нужны регулярные выражения, выделение оглавления, координаты (страница, строка), экспорт CSV. Код должен отличаться по стилю от предыдущего решения.

Результат. DeepSeek предложил более компактный код, в котором регулярные выражения формировались как строки-паттерны, а функции проверки были сведены к небольшому набору предикатов.

Проблема. В первой версии отсутствовала пакетная обработка “каталог/набор файлов”, а также не был унифицирован формат статуса в CSV.

Правка 1. Добавлен отбор входных PDF по “обязательному набору” (если найден), иначе — обработка всех `*.pdf` в каталоге. Формат статуса приведён к точным строкам: **«найдена ошибка» / «ошибка не найдена»**.

Результат правки 1. Реализация стала совместимой по формату вывода с другими версиями, что необходимо для сравнения результатов.

3.2.3 Попытки с Qwen Coder

Промпт 1.

Сгенерируй реализацию варианта проверки структуры PDF в инженерном стиле: вынеси алгоритм в отдельный класс, обеспечь чистый интерфейс, экспорт CSV и обработку каталога. Результат и координаты как в задании.

Результат. Qwen сформировал структуру в виде класса `VariantChecker`, где регулярные выражения и операции выделения оглавления инкапсулированы в методах. CLI реализован минималистично.

Проблема. В первой версии требовалась унификация критериев “первого нарушения”: в некоторых случаях координаты не определялись одинаково по сравнению с другими реализациями.

Правка 1. Приведена логика вычисления координат к общему правилу: если проверка провалена и координату однозначно определить нельзя, используется символ «-». Это обеспечивает корректное сравнение таблиц результатов.

Результат правки 1. Таблицы сравнения стали совпадать по статусам и координатам на тестовом наборе файлов.

3.3 Пример запуска и получаемые файлы

Проверка одного файла:

```
python pdf_checker.py path/to/file.pdf --out results_chatgpt.csv
```

Проверка каталога (если найден обязательный набор, берутся только он; иначе все *.pdf):

```
python pdf_checker.py . --out results_chatgpt.csv
python pdf_checker1_deepseek.py . --out results_deepseek.csv
python pdf_checker2_qwen.py . --out results_qwen.csv
```

Формируемая таблица CSV содержит колонки: `file`, `status`, `first_coord`, где `status` принимает значения **«найдена ошибка»** или **«ошибка не найдена»**.

3.4 Вывод

В этом разделе было описано применяемое программное окружение и структура итоговых программ, а также приведён итеративный процесс разработки с использованием трёх БЯМ: исходные промпты, полученные результаты, выявленные несоответствия и внесённые правки, которые привели к финальным реализациям с единым форматом вывода и возможностью сравнения результатов.

4 Исследовательский раздел

В этом разделе будет выполнено тестирование разработанного программного обеспечения на наборе PDF-файлов, приложенных к лабораторной работе, а также проведено сравнение трёх реализаций, полученных с использованием различных больших языковых моделей (ChatGPT / DeepSeek / Qwen).

Для проверки корректности работы использовался набор PDF-файлов: `_01.pdf ... _10.pdf`, `main (2).pdf`, `ВКР Селез.pdf`. Для каждой из трёх реализаций выполнялся запуск на одном и том же наборе входных файлов, после чего формировался отдельный CSV-файл результатов.

4.1 Форма представления результатов

Для каждого файла фиксировались:

— название PDF-файла;

— признак наличия ошибки (строго: **«найдена ошибка» / «ошибка не найдена»**);

— координаты первого нарушения (страница, строка) либо «-», если координаты определить невозможно (например, если в документе не обнаружен раздел «СОДЕРЖАНИЕ/ОГЛАВЛЕНИЕ»).

4.1.1 Результаты тестирования (реализация ChatGPT / GPT-5.2)

Название PDF	**Признак**	**Координаты первого нарушения**
`_01.pdf`	найдена ошибка	-
`_02.pdf`	найдена ошибка	(2,1)
`_03.pdf`	найдена ошибка	(2,1)
`_04.pdf`	найдена ошибка	(5,1)
`_05.pdf`	найдена ошибка	(1,2)
`_06.pdf`	найдена ошибка	-
`_07.pdf`	найдена ошибка	-
`_08.pdf`	найдена ошибка	(1,1)
`_09.pdf`	найдена ошибка	-
`_10.pdf`	найдена ошибка	(2,1)
`main (2).pdf`	найдена ошибка	(5,1)
`ВКР Селез.pdf`	найдена ошибка	-

4.1.2 Результаты тестирования (реализация DeepSeek DeepThink)

Название PDF	**Признак**	**Координаты первого нарушения**
_01.pdf	найдена ошибка	-
_02.pdf	найдена ошибка	(2,1)
_03.pdf	найдена ошибка	(2,1)
_04.pdf	найдена ошибка	(5,1)
_05.pdf	найдена ошибка	(1,2)
_06.pdf	найдена ошибка	-
_07.pdf	найдена ошибка	-
_08.pdf	найдена ошибка	(1,1)
_09.pdf	найдена ошибка	-
_10.pdf	найдена ошибка	(2,1)
main (2).pdf	найдена ошибка	(5,1)
ВКР Селез.pdf	найдена ошибка	-

4.1.3 Результаты тестирования (реализация Qwen Coder)

Название PDF	**Признак**	**Координаты первого нарушения**
_01.pdf	найдена ошибка	-
_02.pdf	найдена ошибка	(2,1)
_03.pdf	найдена ошибка	(2,1)
_04.pdf	найдена ошибка	(5,1)
_05.pdf	найдена ошибка	(1,2)
_06.pdf	найдена ошибка	-
_07.pdf	найдена ошибка	-
_08.pdf	найдена ошибка	(1,1)
_09.pdf	найдена ошибка	-
_10.pdf	найдена ошибка	(2,1)
main (2).pdf	найдена ошибка	(5,1)
ВКР Селез.pdf	найдена ошибка	-

4.2 Сравнение результатов трёх реализаций

Для сравнения использовались три реализации, разработанные с применением различных БЯМ. Результаты представлены в виде пары: *статус, координаты*.

Файл	**ChatGPT**	**DeepSeek**	**Qwen**
_01.pdf	найдена ошибка, -	найдена ошибка, -	найдена ошибка, -
_02.pdf	найдена ошибка, (2,1)	найдена ошибка, (2,1)	найдена ошибка, (2,1)
_03.pdf	найдена ошибка, (2,1)	найдена ошибка, (2,1)	найдена ошибка, (2,1)
_04.pdf	найдена ошибка, (5,1)	найдена ошибка, (5,1)	найдена ошибка, (5,1)
_05.pdf	найдена ошибка, (1,2)	найдена ошибка, (1,2)	найдена ошибка, (1,2)
_06.pdf	найдена ошибка, -	найдена ошибка, -	найдена ошибка, -
_07.pdf	найдена ошибка, -	найдена ошибка, -	найдена ошибка, -
_08.pdf	найдена ошибка, (1,1)	найдена ошибка, (1,1)	найдена ошибка, (1,1)
_09.pdf	найдена ошибка, -	найдена ошибка, -	найдена ошибка, -
_10.pdf	найдена ошибка, (2,1)	найдена ошибка, (2,1)	найдена ошибка, (2,1)
main (2).pdf	найдена ошибка, (5,1)	найдена ошибка, (5,1)	найдена ошибка, (5,1)
ВКР Селез.pdf	найдена ошибка, -	найдена ошибка, -	найдена ошибка, -

4.3 Оценка сложности

Временная сложность алгоритма линейна по числу строк, извлечённых из PDF-документа, то есть $O(N)$, где N — количество строк после извлечения. Дополнительная память также оценивается как $O(N)$ из-за хранения списка строк с координатами.

4.4 Вывод

По результатам тестирования все три реализации на данном наборе PDF-файлов сформировали одинаковые статусы и совпадающие координаты первого нарушения в случаях, когда координаты можно определить. Следовательно, различий в итоговом результате проверки для выбранного набора входных данных не выявлено.

В этом разделе было выполнено тестирование программ на приложенном наборе PDF-файлов, построены таблицы результатов для каждой реализации и составлена сравнительная таблица, а также дана оценка вычислительной сложности алгоритма.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была достигнута поставленная цель — разработка программного обеспечения для автоматизированной проверки структурных элементов PDF-документов в соответствии с заданным вариантом.

Для достижения цели были решены все поставленные задачи:

— разработаны регулярные выражения для проверки корректности наименований и расположения структурных элементов документа;

— реализована функция извлечения и поиска подстрок в PDF-файлах с использованием регулярных выражений;

— разработано программное обеспечение, принимающее на вход путь к отдельному PDF-файлу либо к каталогу с файлами и выполняющее их пакетную обработку;

— проведено тестирование программы на наборе PDF-документов, приложенных к лабораторной работе;

— выполнено сравнение решений, полученных с использованием различных больших языковых моделей.

Результаты тестирования подтвердили корректность работы разработанного программного обеспечения и его соответствие требованиям задания.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. PyPDF2 Documentation [Электронный ресурс] // PyPI : [сайт]. — Режим доступа: https://pypi.org/project/PyPDF2/ (дата обращения: 12.12.2025).

2. Python 3 Documentation [Электронный ресурс] // Python Software Foundation : [сайт]. — Режим доступа: https://docs.python.org/3/ (дата обращения: 12.12.2025).

3. Regular Expression HOWTO [Электронный ресурс] // Python Software Foundation : [сайт]. — Режим доступа: https://docs.python.org/3/howto/regex.html (дата обращения: 12.12.2025).

4. Ollama [Электронный ресурс] // Ollama : [сайт]. — Режим доступа: https://ollama.com/ (дата обращения: 11.12.2025).

5. Qwen (модели семейства Qwen, включая coder-ориентированные) [Электронный ресурс] // QwenLM (GitHub) : [сайт]. — Режим доступа: https://github.com/QwenLM/Qwen (дата обращения: 12.12.2025).

6. DeepSeek [Электронный ресурс] // DeepSeek : [сайт]. — Режим доступа: https://www.deepseek.com/ (дата обращения: 12.12.2025).

7. ChatGPT [Электронный ресурс] // OpenAI : [сайт]. — Режим доступа: https://chat.openai.com/ (дата обращения: 12.12.2025).

ПРИЛОЖЕНИЕ А

Листинг А.1 — Извлечение строк PDF и формирование базовых шаблонов (модель ChatGPT)

```
from __future__ import annotations

import re
from typing import List, Tuple
from PyPDF2 import PdfReader

Line = Tuple[str, int, int]

def spaced_letters(word: str) -> str:
    return r"\s*".join(re.escape(ch) for ch in word)

W_TOC_1 = "SODERZHANIE"
W_TOC_2 = "OGLAVLENIE"
W_INTRO = "VVEDENIE"
W_CONCL = "ZAKLYUCHENIE"
W_LIST  = "SPISOK"
W_USED  = "ISPOLSOVANNYH"
W_SRC   = "ISTOCHNIKOV"

RX_TOC = re.compile(rf"^\s*(?:{W_TOC_1}|{W_TOC_2})\s*$", re.IGNORECASE)
RX_INTRO = re.compile(rf"^\s*{spaced_letters(W_INTRO)}\b", re.
   IGNORECASE)
RX_CONCL = re.compile(rf"^\s*{spaced_letters(W_CONCL)}\b", re.
   IGNORECASE)
RX_REFS = re.compile(
    rf"^\s*{spaced_letters(W_LIST)}\s+"
    rf"{spaced_letters(W_USED)}\s+"
    rf"{spaced_letters(W_SRC)}\b",
    re.IGNORECASE
)

def extract_lines(pdf_path: str) -> List[Line]:
    reader = PdfReader(pdf_path)
    result: List[Line] = []

    for page_no, page in enumerate(reader.pages, start=1):
        text = page.extract_text() or ""
        for line_no, raw in enumerate(text.splitlines(), start=1):
            line = raw.strip()
```

```
            if line:
                result.append((line, page_no, line_no))

    return result
```

Листинг А.2 — Определение границ оглавления и проверка структуры (модель DeepSeek)

```
import re
from typing import List, Optional, Tuple

Line = Tuple[str, int, int]
Match = Tuple[str, Tuple[int, int]]

RX_TOC_ENTRY = re.compile(r"^(?=.*[A-Za-z])(?=.*\d).+\d+$")

def toc_bounds(lines: List[Line], limit: int = 500) -> Tuple[int, int]:
    start = next(
        (i for i, (text, _, _) in enumerate(lines) if RX_TOC.match(text
           )),
        -1
    )
    if start < 0:
        return -1, -1

    end = start + 1
    miss = 0

    for i in range(start + 1, min(len(lines), start + limit)):
        if RX_TOC_ENTRY.match(lines[i][0]):
            end = i + 1
            miss = 0
        else:
            miss += 1
            if miss >= 5:
                break

    return start, end

def first_match(where: List[Line], rx: re.Pattern) -> Optional[Line]:
    for text, page, line in where:
        if rx.search(text):
            return (text, page, line)
    return None

def check_structure(lines: List[Line]) -> Tuple[bool, List[Match]]:
    a, b = toc_bounds(lines)
    if a < 0:
```

```
        return False, []

    toc = lines[a:b]
    body = lines[:a] + lines[b:]

    intro = first_match(toc, RX_INTRO)
    concl = first_match(toc, RX_CONCL)
    refs  = first_match(toc, RX_REFS)

    if not (intro and concl and refs):
        return False, []

    def heading_rx(rx: re.Pattern) -> re.Pattern:
        return re.compile(
            rf"^\s*(?:\d+(?:\.\d+)*)?\s*({rx.pattern})\s*$",
            re.IGNORECASE
        )

    if (first_match(body, heading_rx(RX_INTRO)) or
        first_match(body, heading_rx(RX_CONCL)) or
        first_match(body, heading_rx(RX_REFS))):
        return False, []

    return True, [
        (intro[0], (intro[1], intro[2])),
        (concl[0], (concl[1], concl[2])),
        (refs[0],  (refs[1],  refs[2])),
    ]
```

Листинг А.3 — Оптимизация досрочного завершения проверки (модель Qwen)

```
from typing import Dict, List, Optional, Tuple

Line = Tuple[str, int, int]
Match = Tuple[str, Tuple[int, int]]

def find_required_in_toc(toc: List[Line]) -> Optional[List[Match]]:
    found: Dict[str, Optional[Match]] = {
        "intro": None,
        "concl": None,
        "refs": None
    }

    for text, page, line in toc:
        if found["intro"] is None and RX_INTRO.search(text):
            found["intro"] = (text, (page, line))
        elif found["concl"] is None and RX_CONCL.search(text):
            found["concl"] = (text, (page, line))
        elif found["refs"] is None and RX_REFS.search(text):
            found["refs"] = (text, (page, line))

        if all(found.values()):
            return [
                found["intro"],
                found["concl"],
                found["refs"]
            ]

    return None
```